

Name - Suparba Sadhukhon

Roll no - UG1/04/BTCE 12025/037

1) (b) head and pointer to the next node .

2) (c) NULL

3) (b) self referential structure .

4. (b) head == NULL

5. (b) newnode $\rightarrow$ next = head; head = newnode;

6. (d) ~~head~~ temp = head $\rightarrow$ next; free(head); head = temp $\rightarrow$ next;

7. (c) Traversing from tail to head .

8. (a) n-1 .

9. (c) Copying the data of p $\rightarrow$ next into p, then deleting p next .

10. (a) while (p $\rightarrow$ next != NULL)

11. (b) p $\rightarrow$ next == NULL

12. (c)

13. (b) Two .

14. (a) Zero nodes .

15. (b) p = (struct node *) malloc (sizeof (struct node));

16. (c)

17. (c) the first node .

18. (b)

19. (b) Loss of access to all nodes, causing a memory leak .

20. (b) a slow pointer and a fast pointer .

21. (c) The last node .

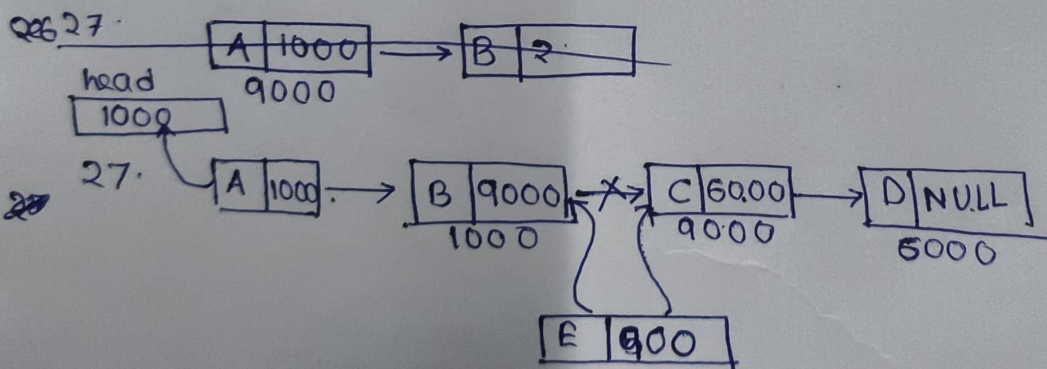
22. (b) Nodes may be scattered anywhere in memory .

23. struct node
{
int data;
struct node *next;
};

24. 30 40 70

25. ~~check~~ There should a if-else condition to check whether the list is empty or list has the nodes

```
void insertEnd(struct node * head, int x) {
    struct node * t = (struct node *) malloc(sizeof(struct node));
    t->data = x;
    t->next = NULL;
    struct node * p = head;
    if (p != NULL)
    {
        while (p->next != NULL)
            p = p->next;
        p->next = t;
    }
    else
        head = t;
    return head;
}
```



28. 5 → 8 → 12 → 3 → NULL

(i) Insert 7 at the beginning.

7 → 5 → 8 → 12 → 3 → NULL

(ii) Delete the node containing 12.

7 → 5 → 8 → 3

At first we remove 12 and connect its previous node 8 → 3 (next node).

(iii) Insert 9 at the end.

7 → 5 → 8 → 3 → 9 → NULL